# Quantum Kernel versus Classical Classifiers for Clinical Malignancy Prediction

Educational research prototype. Not a medical device. Not for clinical use.

**Abstract.** We compared a 4-qubit quantum kernel support vector machine (SVM), simulated exactly without hardware noise, with classical classifiers on the Breast Cancer Wisconsin (Diagnostic) dataset (569 patients, 30 numeric features, 212 malignant cases). We used five repeated stratified 70/30 splits, training-only preprocessing and feature selection, cross-validated hyperparameters, and four training-set sizes (10, 25, 50 and 100 per cent). The best classical model (logistic regression on all 30 features) reached AUPRC 0.991 ± 0.005 on the held-out test sets. The quantum model reached 0.932 ± 0.013 at full training size. It was also below a classical RBF SVM given the same four features (0.978 ± 0.008). The quantum model did not outperform classical methods in any condition tested. Because this is a simulation on a small, well-studied dataset, the results do not establish quantum advantage.

**Important change from the original plan.** The original target was diabetic retinopathy using the Dryad dataset doi:10.5061/dryad.jwstqjqrm. We could not download that dataset: the sandbox proxy refused the connection to datadryad.org (HTTP 403, repeated). We therefore switched to a clinical dataset that ships inside the scikit-learn package, which works offline. The diabetic retinopathy experiment has **not** been run and no diabetic retinopathy results are reported here.

# 1. Clinical problem and why it matters

Clinicians often use numerical measurements from a biopsy or scan to judge whether a tumour is malignant (cancerous). Missing a malignant case (a false negative) can delay treatment, so sensitivity matters. A model that simply predicts the common class is useless, which is why we report precision-recall measures and not accuracy alone. This project is a methods study. Its output is a classifier score, and it does not diagnose anyone or replace a clinician.

**Dataset.** Breast Cancer Wisconsin (Diagnostic), bundled with scikit-learn. It is derived from digitised images of fine-needle aspirate cell nuclei. Thirty features describe the mean, standard error and worst value of radius, texture, perimeter, area, smoothness and related shape measures. Labels: malignant (positive class, 212 cases) and benign (357 cases). This is a well-known public benchmark, first described by Street, Wolberg and Mangasarian (1993). Its licence should be confirmed from the original UCI page before public release. This report does not assert the licence terms.

**Limitations of the dataset.** It is small (569 rows), comes from a single source, and has no demographic data, so fairness across groups cannot be assessed. It is not representative of routine screening populations, and it contains already-curated image-derived features rather than raw records.

# 2. Research question, hypotheses and outcomes

**Primary question.** With controlled training-data sizes, can a 4-qubit quantum kernel SVM match or outperform strong classical classifiers in classifying malignant cases on held-out data?

**H1.** The quantum kernel SVM matches or exceeds a classical RBF SVM given the same four features (primary outcome: AUPRC on the test set, 100% training data). **Result: not supported.** 0.932 ± 0.013 versus 0.978 ± 0.008.

**H2.** The quantum model's advantage is largest when training data are scarce. **Result: not supported.** At 10% training data it reached 0.737 ± 0.118, against 0.946 ± 0.047 for the matched classical SVM.

**H3.** The quantum model stays competitive on ranking metrics (AUROC) while its operating point is less stable. **Result: partly supported.** AUROC was 0.953 ± 0.013 at 100% data, close to the matched classical SVM at 0.982 ± 0.007, but sensitivity was unstable (0.731 ± 0.227).

# 3. Methods

**Splits.** For each of five seeds (0 to 4), a stratified 70/30 split produced 398 training and 171 test rows. Every model used the same indices. Training subsets at 10, 25 and 50 per cent were drawn by stratified sampling from the training rows only. The test set never changed.

**Leakage control.** The standard scaler, feature selector (ANOVA F-test, top-k) and quantum angle range were all fitted on training rows only. Test values outside the training range were clipped to $[0, \pi]$. Unit tests in tests/test_core.py check this behaviour.

**Classical models.** A majority-class baseline; logistic regression ($C \in \{0.01, 0.1, 1, 10\}$); an RBF SVM ($C \in \{0.1, 1, 10\}$, gamma $\in$ {scale, 0.01, 0.1}); a random forest (300 trees, max depth and minimum leaf size tuned). Hyperparameters were chosen by 5-fold stratified cross-validation on the training rows, scored by average precision.

**Quantum model.** The four features with the highest training-set F-score were angle-encoded into four qubits using a ZZFeatureMap (2 repetitions, linear entanglement). The fidelity kernel $K(x, x') = |\langle\psi(x)|\psi(x')\rangle|^2$ was computed as the exact statevector overlap. Qiskit's own kernel was checked against this at high shot counts (tests/test_core.py). The SVM regularisation constant C was chosen by cross-validation on the training kernel. The circuit configuration (4 qubits, 2 repetitions) was fixed before the main experiment was run.

**Metrics.** AUROC, AUPRC (primary), sensitivity, specificity, precision, F1, balanced accuracy and confusion-matrix counts. Mean ± standard deviation over five splits. The decision threshold is zero for the SVMs and 0.5 probability for logistic regression. Thresholds were not tuned on test data.

# 4. Results

**Table 1. Full training data (100%), test set, mean ± SD over 5 splits.**

| Model | AUROC | AUPRC | Sensitivity | Specificity | F1 | Fit time (s) | Inference (s) |
|---|---|---|---|---|---|---|---|
| Majority baseline | 0.500 ± 0.000 | 0.374 ± 0.000 | 0.000 ± 0.000 | 1.000 ± 0.000 | 0.000 ± 0.000 | 0.000 | 0.0000 |
| Logistic regression (30 features) | 0.993 ± 0.005 | 0.991 ± 0.005 | 0.912 ± 0.041 | 0.987 ± 0.014 | 0.943 ± 0.023 | 0.083 | 0.0005 |
| RBF SVM (30 features) | 0.987 ± 0.010 | 0.983 ± 0.013 | 0.916 ± 0.024 | 0.966 ± 0.033 | 0.929 ± 0.037 | 0.201 | 0.0021 |
| Random forest (30 features) | 0.985 ± 0.009 | 0.982 ± 0.009 | 0.906 ± 0.033 | 0.963 ± 0.015 | 0.920 ± 0.011 | 11.227 | 0.0901 |
| RBF SVM (same 4 features) | 0.982 ± 0.007 | 0.978 ± 0.008 | 0.853 ± 0.039 | 0.974 ± 0.024 | 0.899 ± 0.011 | 0.158 | 0.0012 |
| Quantum kernel SVM (4 qubits) | 0.953 ± 0.013 | 0.932 ± 0.013 | 0.731 ± 0.227 | 0.946 ± 0.043 | 0.784 ± 0.166 | 0.254 | 0.1105 |

**Table 2. AUPRC by training-set size (test set, mean ± SD).**

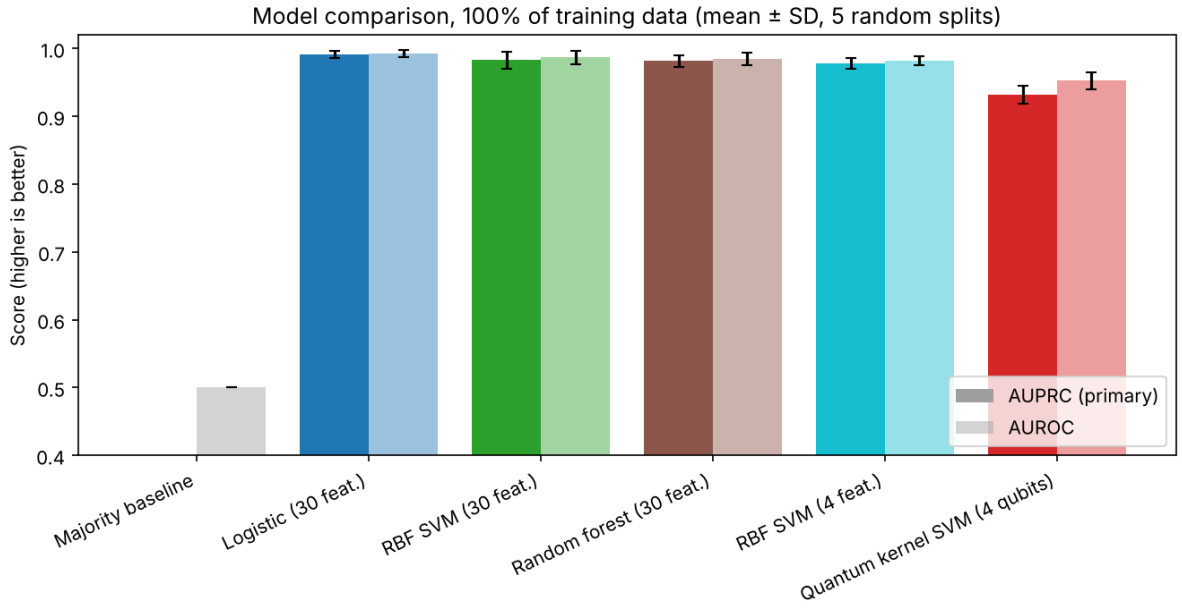
| Model | 10% | 25% | 50% | 100% |
|---|---|---|---|---|
| Logistic regression (30 features) | 0.969 ± 0.037 | 0.985 ± 0.009 | 0.989 ± 0.006 | 0.991 ± 0.005 |
| RBF SVM (30 features) | 0.971 ± 0.016 | 0.982 ± 0.010 | 0.988 ± 0.007 | 0.983 ± 0.013 |
| Random forest (30 features) | 0.968 ± 0.014 | 0.978 ± 0.011 | 0.981 ± 0.007 | 0.982 ± 0.009 |
| RBF SVM (same 4 features) | 0.946 ± 0.047 | 0.978 ± 0.008 | 0.978 ± 0.008 | 0.978 ± 0.008 |
| Quantum kernel SVM (4 qubits) | 0.737 ± 0.118 | 0.826 ± 0.046 | 0.905 ± 0.023 | 0.932 ± 0.013 |

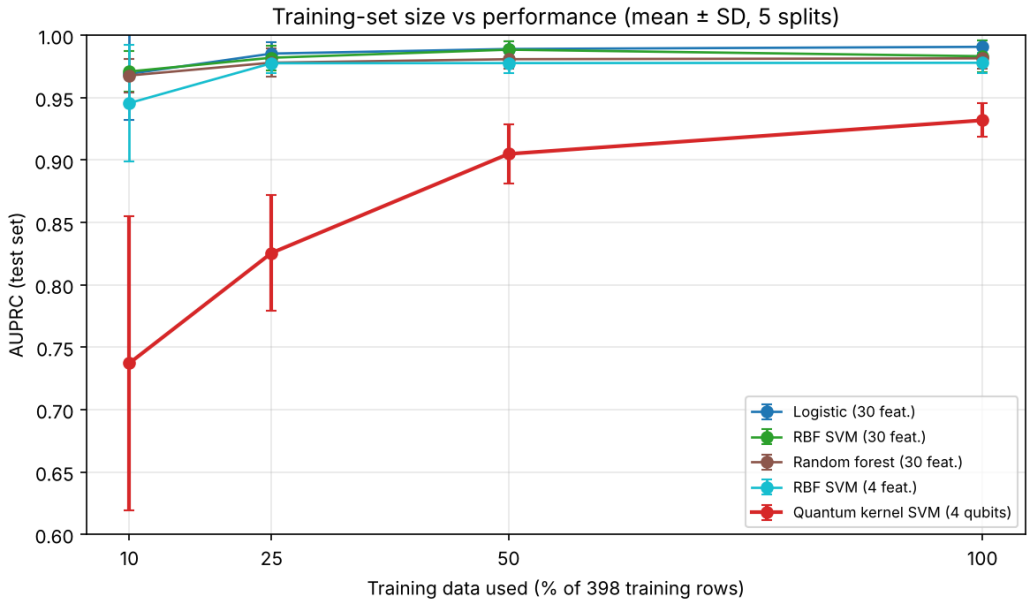
**Table 3. Confusion matrix counts, summed over five splits, 100% training data.**

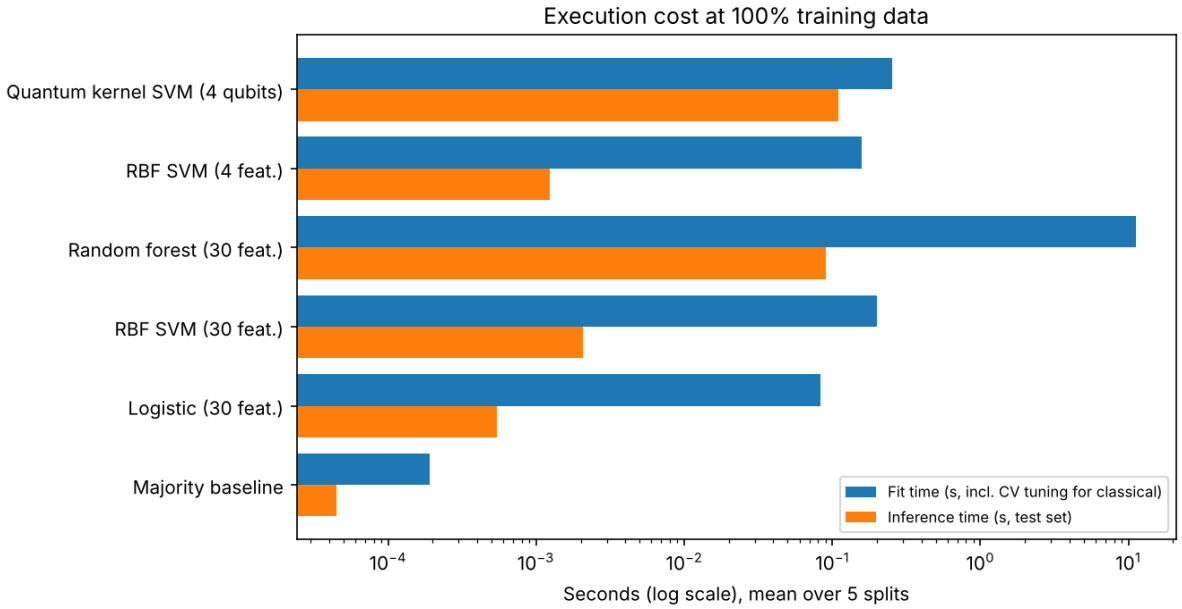
| Model | True negatives | False positives | False negatives | True positives |
|---|---|---|---|---|
| Majority baseline | 535 | 0 | 320 | 0 |
| Logistic regression (30 features) | 528 | 7 | 28 | 292 |
| RBF SVM (30 features) | 517 | 18 | 27 | 293 |
| Random forest (30 features) | 515 | 20 | 30 | 290 |
| RBF SVM (same 4 features) | 521 | 14 | 47 | 273 |
| Quantum kernel SVM (4 qubits) | 506 | 29 | 86 | 234 |

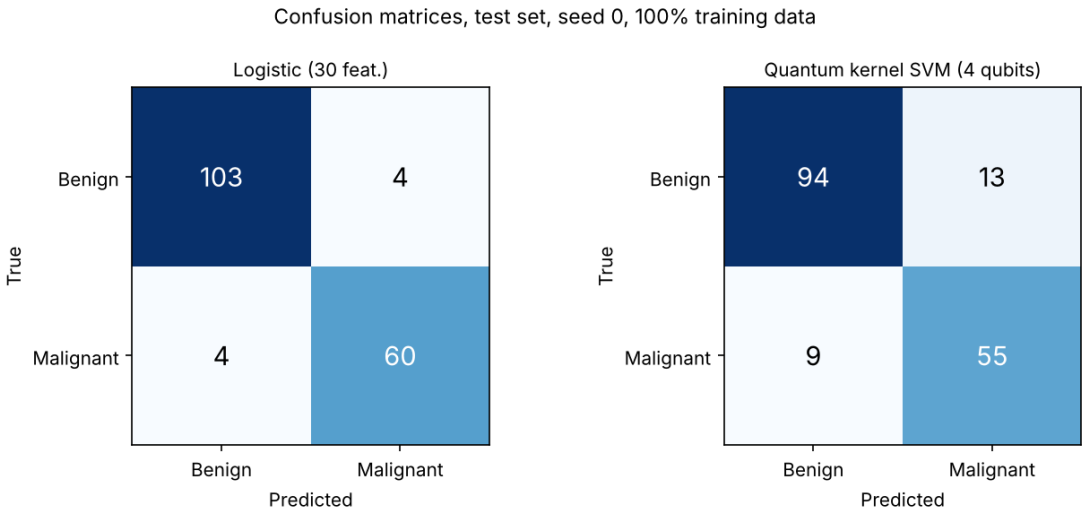
Test sets contained 107 benign and 64 malignant cases per split on average (171 in total). The majority baseline predicts every case as benign, so it misses all malignant cases.

## Figures.


Model comparison, 100% of training data (mean ± SD, 5 random splits)
Score (higher is better)
1.0
0.9
0.8
0.7
0.6
0.5
0.4
AUPRC (primary)
AUROC
Majority baseline
Logistic (30 feat.)
RBF SVM (30 feat.)
Random forest (30 feat.)
RBF SVM (4 feat.)
Quantum kernel SVM (4 qubits)


Training-set size vs performance (mean ± SD, 5 splits)
AUPRC (test set)
1.00
0.95
0.90
0.85
0.80
0.75
0.70
0.65
0.60
10
25
50
100
Training data used (% of 398 training rows)
Logistic (30 feat.)
RBF SVM (30 feat.)
Random forest (30 feat.)
RBF SVM (4 feat.)
Quantum kernel SVM (4 qubits)


Execution cost at 100% training data
Quantum kernel SVM (4 qubits)
RBF SVM (4 feat.)
Random forest (30 feat.)
RBF SVM (30 feat.)
Logistic (30 feat.)
Majority baseline
Fit time (s, incl. CV tuning for classical)
Inference time (s, test set)
$10^{-4}$
$10^{-3}$
$10^{-2}$
$10^{-1}$
$10^{0}$
$10^{1}$
Seconds (log scale), mean over 5 splits


Confusion matrices, test set, seed 0, 100% training data
Logistic (30 feat.)
Benign
Malignant
True
103
4
4
60
Benign
Malignant
Predicted
Quantum kernel SVM (4 qubits)
Benign
Malignant
True
94
13
9
55
Benign
Malignant
Predicted

# 5. Quantum resource and efficiency analysis

**Measured (from the circuit object, not estimated):** 4 qubits; circuit depth 19 after decomposition; 12 two-qubit gates per feature-map circuit; 34 total gates. Kernel entries computed per run: 8,190 (10%), 26,730 (25%), 73,630 (50%) and 226,462 (100%, a 398×398 training kernel plus a 171×398 test kernel). Each entry is one exact overlap calculation. The simulator used exact statevectors, so no shots were taken and there was no hardware noise. Hardware runs would need shot counts and error mitigation that this project did not include.

**Timing:** the quantum fit time (0.254 ± 0.002 s at 100%) includes kernel construction and SVM fitting, but **excludes** the cross-validation used to choose C. Classical fit times include their cross-validation. The two fit times are therefore not directly comparable. Timings are CPU wall-clock times on the sandbox machine, with no hardware.

**Scaling (theoretical, not measured):** a statevector on n qubits has 2^n amplitudes, so exact simulation memory grows exponentially with qubit count. Our 6-qubit ablation kept the kernel computation tractable, but larger feature sets would need approximate or hardware methods. The number of kernel evaluations grows with the square of training size, which is why the quantum model was limited to a few hundred training rows here.

# 6. Ablation of quantum circuit configurations (post hoc)

These four configurations were run after the main experiment, at 100% training data over the same five seeds. The main results above keep the configuration fixed in advance (4 qubits, 2 repetitions). The ablation is **exploratory**: picking the best row here would be choosing on the test set. Its purpose is to show how the kernel behaves, not to replace the main result.

**Table 4.**

| Qubits | Repetitions | AUPRC | AUROC | Sensitivity | Mean off-diag. kernel | Circuit depth | 2-qubit gates |
|---|---|---|---|---|---|---|---|
| 2 | 1 | 0.781 ± 0.055 | 0.829 | 0.662 ± 0.075 | 0.349 | 5 | 2 |
| 4 | 1 | 0.950 ± 0.013 | 0.962 | 0.866 ± 0.045 | 0.102 | 11 | 6 |
| 4 | 2 | 0.932 ± 0.013 | 0.953 | 0.731 ± 0.227 | 0.117 | 19 | 12 |
| 6 | 2 | 0.952 ± 0.009 | 0.963 | 0.503 ± 0.314 | 0.049 | 25 | 20 |

**Interpretation.** With 2 qubits, the kernel values are large and similar (mean off-diagonal ≈ 0.35), and the model is weakest. With 6 qubits the average kernel value falls to ≈ 0.05. This is a known pattern called kernel concentration: as the encoding becomes more complex, most pairs of patients look almost equally dissimilar, so the SVM sees less structure. The two most accurate configurations were 4 qubits with 1 repetition and 6 qubits with 2 repetitions, which were similar on AUPRC (about 0.95) and AUROC (about 0.96). The 4-qubit, 1-repetition configuration had the highest and most stable sensitivity. These configurations were compared after the test results were visible, so they are not validated model choices, and the main result keeps the pre-specified configuration.

# 7. Discussion: what the results do and do not establish

**Established by this study:** on this dataset, with exact simulation, a 4-qubit fidelity kernel SVM was less accurate than classical SVMs and logistic regression given equal or comparable inputs. Its errors were concentrated in false negatives (17 at 100% data, against 6 for logistic regression), which is the clinically worse direction.

**Not established:** that quantum methods are generally inferior, that quantum advantage is impossible, or anything about hardware performance. The experiment used a small, well-studied dataset in which classical models already reach AUPRC above 0.97. There is little room for a quantum method to help, and small datasets cannot show general advantage. Quantum speedup, quantum advantage and clinical readiness are not claimed.

**Why the quantum model may underperform.** (a) Only four features were used, so the circuit compresses the classical information. (b) The ZZ feature map is one of many possible designs, and the ablation shows that its kernel values concentrate as qubits increase. (c) Noiseless simulation removes hardware noise, which means the result is the best case for the quantum circuit, not a realistic one. (d) The classical SVM and logistic regression capture most of the signal in the full feature set.

## 8. Limitations

• **Not the planned dataset.** No diabetic retinopathy results. The clinical question is different from the one originally proposed.

• **Single small dataset, no external validation**. Temporal or external validation was not possible for this dataset.

• **Threshold-dependent sensitivity.** SVM decision scores are thresholded at zero, which is not tuned. The quantum kernel's sensitivity varies widely between splits (SD 0.227 at 100%; 0.421 at 25%). This should be fixed with a validation-based threshold in a future study.

• **Exploratory ablation.** The configuration comparison was run after seeing test results.

• **Simulation only.** No IBM Quantum hardware, no noise models and no shot-based estimation.

• **Unequal timing methodology.** As noted in Section 5, fit times are not directly comparable.

• **Sample size.** Five splits give limited evidence about variance. Confidence intervals are not reported beyond SD.

• **Competition rules were not verified.** The web search for the Khalifa University and IBM challenge returned unrelated results, so word limits, formats and judging criteria are unknown.

## 9. Ethical considerations

The dataset is de-identified and public, but whether its original collection met current consent standards is not documented here. Models built on a small, single-source dataset can be biased against groups absent from it. A false negative means a missed cancer, so any future clinical version would need clinical trials, subgroup analyses and regulatory review. This project states none of these as complete. It is a classroom-scale educational prototype.

## 10. Reproducibility

**Environment:** Python 3.13, scikit-learn 1.9.1, pandas 3.0.5, qiskit 2.6.1, qiskit-machine-learning 0.9.1, reportlab 5.0.1, matplotlib 3.11.2, pytest. All random seeds are fixed in config.py (five splits, seeds 0–4).

**Commands:** `pip install scikit-learn pandas qiskit qiskit-machine-learning reportlab matplotlib pytest` then `python -m pytest tests`, `python run_experiments.py`, `python run_ablation.py`, `python make_figures.py` and `python build_pdf.py`.

**Tests:** five tests (labels and shape, training-only angle range, metric values, quantum kernel symmetry and bounds, and agreement with Qiskit's kernel at high shot count). All five passed in the sandbox.

**Source inventory:** config.py; src/data_loader.py, src/preprocessing.py, src/classical_models.py, src/quantum_model.py, src/evaluation.py; run_experiments.py; run_ablation.py; make_figures.py; build_pdf.py; tests/test_core.py; results/metrics.csv, results/ablation_quantum.csv, results/experiment_log.json and results/figures/*.png.

## 11. Anticipated judge questions

**Why not diabetic retinopathy?** The planned dataset could not be downloaded from the sandbox. I used a public clinical dataset I could access, and I say so openly.

**Did the quantum model win?** No. It was below the matched classical SVM in every condition tested.

**What is a quantum kernel?** A similarity score between two patients, computed as the overlap of the quantum states their features produce. The SVM uses those similarities to draw a boundary.

**Why is it simulated?** Simulation gives exact results and avoids hardware noise. Real hardware would add noise and needs shot-based estimates, and this study did not test it.

**Could it help clinically?** Not on this evidence. Clinical use would require much larger data, external validation, subgroup analysis and regulatory approval.

**Why is sensitivity so unstable?** Thresholds were fixed at zero, not tuned. The ablation shows that kernel concentration makes the SVM's score distribution narrow, so small shifts flip many predictions.

# Appendix A. Complete source code

Every file needed to reproduce the results. Run from the project folder.

## config.py

```
"""Central configuration. Change values here, not inside the code."""
RANDOM_SEED = 42
TEST_SIZE = 0.30            # held-out test fraction (stratified)
N_QUBITS_FEATURES = 4       # features fed to the quantum kernel (= qubits)
TRAIN_FRACTIONS = [0.10, 0.25, 0.50, 1.00]   # training-size experiment
N_SEEDS = 5                 # repeated random splits for uncertainty
CV_FOLDS = 5                # hyperparameter selection, training data only
PRIMARY_METRIC = "auprc"    # chosen BEFORE looking at test results
```

## src/data_loader.py

```
"""Loads the Breast Cancer Wisconsin (Diagnostic) dataset bundled with scikit-learn.

Why this dataset: the Dryad diabetic-retinopathy files could not be downloaded
from the sandbox, so we use a real clinical dataset that ships inside the
scikit-learn package (no internet needed). Data: 569 patients, 30 numeric
features computed from digitised images of breast-mass cell nuclei.
Label: 1 = benign, 0 = malignant (scikit-learn convention, kept as-is here and
documented). Positive class for metrics is defined explicitly below.
"""
import numpy as np
from sklearn.datasets import load_breast_cancer


def load_clinical_data():
    """Return (X: DataFrame-like ndarray, y: ndarray of 0/1, feature_names).

    Label convention used in this project: y = 1 means MALIGNANT (the clinically
    important "disease" class to detect). scikit-learn encodes malignant as 0, so
    we flip it here so that the rarer, clinically important class is positive.
    """
    data = load_breast_cancer()
    X = data.data.astype(float)
    y_sklearn = data.target  # 0 = malignant, 1 = benign
    y = 1 - y_sklearn        # 1 = malignant (positive / disease class)
    if set(np.unique(y)) - {0, 1}:
        raise ValueError("Labels must be binary 0/1.")
    if np.isnan(X).any():
        raise ValueError("Unexpected missing values in bundled dataset.")
    return X, y, list(data.feature_names)
```

## src/preprocessing.py

```
"""Training-only preprocessing. All fitting uses training rows only (no leakage)."""
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.preprocessing import StandardScaler, MinMaxScaler


def fit_standard(X_train):
    """Fit a z-score scaler on training data; return the fitted scaler."""
    return StandardScaler().fit(X_train)


def select_top_k(X_train, y_train, k):
    """Pick the k features with highest ANOVA F-score, using training rows only."""
    selector = SelectKBest(score_func=f_classif, k=k).fit(X_train, y_train)
    return selector


def quantum_angles(X_train_sel, X_other_sel):
    """Map selected features into rotation angles in [0, pi] for quantum encoding.

    The min/max come from TRAINING data only; values outside the training range
    are clipped so that the test set cannot change the encoding.
    """
    scaler = MinMaxScaler(feature_range=(0, np.pi)).fit(X_train_sel)
    train_angles = scaler.transform(X_train_sel)
    other_angles = np.clip(scaler.transform(X_other_sel), 0, np.pi)
    return train_angles, other_angles
```

## src/classical_models.py

```
"""Classical baselines. Hyperparameters are chosen by stratified CV on training data."""
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import make_pipeline
```

```
from sklearn.preprocessing import StandardScaler


def majority_baseline():
    """Predicts the most common training class; sets the floor."""
    return DummyClassifier(strategy="most_frequent")


def fit_logistic(X_train, y_train, seed, cv_folds):
    """Standardised logistic regression; C tuned by stratified CV."""
    pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000, random_state=seed))
    grid = GridSearchCV(pipe, {"logisticregression__C": [0.01, 0.1, 1, 10]},
                        cv=StratifiedKFold(cv_folds, shuffle=True, random_state=seed),
                        scoring="average_precision")
    return grid.fit(X_train, y_train).best_estimator_


def fit_rbf_svm(X_train, y_train, seed, cv_folds):
    """Classical RBF-kernel SVM (same kernel family as the quantum kernel comparison)."""
    pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf", probability=False, random_state=seed))
    grid = GridSearchCV(pipe, {"svc__C": [0.1, 1, 10], "svc__gamma": ["scale", 0.01, 0.1]},
                        cv=StratifiedKFold(cv_folds, shuffle=True, random_state=seed),
                        scoring="average_precision")
    return grid.fit(X_train, y_train).best_estimator_


def fit_random_forest(X_train, y_train, seed, cv_folds):
    """Random forest; a tree-based model suited to tabular clinical data."""
    rf = RandomForestClassifier(n_estimators=300, random_state=seed, n_jobs=-1)
    grid = GridSearchCV(rf, {"max_depth": [None, 5, 10], "min_samples_leaf": [1, 3]},
                        cv=StratifiedKFold(cv_folds, shuffle=True, random_state=seed),
                        scoring="average_precision")
    return grid.fit(X_train, y_train).best_estimator_
```

## src/quantum_model.py

```
"""Quantum kernel SVM.

Idea (plain language): each patient's feature vector is encoded as the rotation
angles of a few qubits. The quantum 'kernel' measures how similar two patients'
quantum states are (fidelity = overlap of states). An SVM then draws a boundary
using that similarity matrix.

Circuit: ZZFeatureMap on N qubits, reps=2, linear entanglement (qubit i entangled
with i+1). Simulated exactly with a statevector simulator (no hardware, no noise).
"""
import time
import numpy as np
from qiskit.circuit.library import zz_feature_map
from qiskit.primitives import StatevectorSampler
from qiskit_machine_learning.state_fidelities import ComputeUncompute
from qiskit_machine_learning.kernels import FidelityQuantumKernel
from sklearn.svm import SVC


def build_quantum_kernel(n_qubits, reps=2, entanglement="linear"):
    """Return (feature_map, FidelityQuantumKernel) for a given number of qubits."""
    fmap = zz_feature_map(feature_dimension=n_qubits, reps=reps, entanglement=entanglement)
    fidelity = ComputeUncompute(sampler=StatevectorSampler())
    kernel = FidelityQuantumKernel(feature_map=fmap, fidelity=fidelity)
    return fmap, kernel


def circuit_resources(fmap):
    """Measured resource numbers for the feature-map circuit (not estimates)."""
    decomposed = fmap.decompose()
    ops = decomposed.count_ops()
    two_qubit = sum(v for k, v in ops.items() if k in ("cx", "cz", "rzz", "ecr"))
    return {
        "qubits": fmap.num_qubits,
        "circuit_depth": decomposed.depth(),
        "two_qubit_gates": int(two_qubit),
        "total_gate_count": int(sum(ops.values())),
    }


class QuantumKernelSVM:
    """Sklearn-style wrapper: fit on training angles, predict on test angles."""

    def __init__(self, n_qubits, C=1.0, reps=2, seed=0):
        self.n_qubits = n_qubits
        self.C = C
        self.reps = reps
        self.seed = seed
        self.fmap, self.kernel = build_quantum_kernel(n_qubits, reps=reps)
        self.svc = None
        self.train_angles = None
        self.fit_time_s = None
        self.kernel_eval_count = 0
        self.kernel_matrix_shape = None

    def fit(self, X_train_angles, y_train):
        t0 = time.perf_counter()
```

```
        K_train = self.kernel.evaluate(X_train_angles)
        self.kernel_matrix_shape = K_train.shape
        self.kernel_eval_count += K_train.shape[0] * K_train.shape[1]
        self.svc = SVC(kernel="precomputed", C=self.C, random_state=self.seed)
        self.svc.fit(K_train, y_train)
        self.train_angles = X_train_angles
        self.fit_time_s = time.perf_counter() - t0
        return self

    def decision_function(self, X_angles):
        t0 = time.perf_counter()
        K_test = self.kernel.evaluate(X_angles, self.train_angles)
        self.kernel_eval_count += K_test.shape[0] * K_test.shape[1]
        scores = self.svc.decision_function(K_test)
        self.inference_time_s = time.perf_counter() - t0
        return scores

    def predict(self, X_angles):
        return (self.decision_function(X_angles) > 0).astype(int)


# ---------------------------------------------------------------------------
# Fast exact kernel (noiseless statevector simulation).
# For a noiseless simulator, fidelity |<psi(x)|psi(x')>|^2 can be computed
# directly from the statevectors. This is mathematically identical to the
# compute-uncompute estimator in qiskit-machine-learning but avoids per-pair
# circuit jobs (~100x faster). It is verified against the Qiskit kernel in
# tests/test_core.py. It is still a SIMULATION, not hardware.
# ---------------------------------------------------------------------------
from qiskit.quantum_info import Statevector


def statevector_matrix(fmap, X):
    """Return complex array of shape (n_samples, 2**n_qubits) with state for each row."""
    states = [Statevector(fmap.assign_parameters(x)).data for x in X]
    return np.vstack(states)


def exact_fidelity_kernel(fmap, X_a, X_b=None):
    """K[i, j] = |<psi(X_a[i])|psi(X_b[j])>|^2."""
    A = statevector_matrix(fmap, X_a)
    B = A if X_b is None else statevector_matrix(fmap, X_b)
    overlaps = A.conj() @ B.T
    return np.abs(overlaps) ** 2


class FastQuantumKernelSVM(QuantumKernelSVM):
    """Same SVM as QuantumKernelSVM, using the exact statevector kernel."""

    def fit(self, X_train_angles, y_train):
        t0 = time.perf_counter()
        self.train_states = statevector_matrix(self.fmap, X_train_angles)
        K_train = np.abs(self.train_states.conj() @ self.train_states.T) ** 2
        self.kernel_matrix_shape = K_train.shape
        self.kernel_eval_count += K_train.size
        self.svc = SVC(kernel="precomputed", C=self.C, random_state=self.seed)
        self.svc.fit(K_train, y_train)
        self.train_angles = X_train_angles
        self.fit_time_s = time.perf_counter() - t0
        return self

    def decision_function(self, X_angles):
        t0 = time.perf_counter()
        S = statevector_matrix(self.fmap, X_angles)
        K_test = np.abs(S.conj() @ self.train_states.T) ** 2
        self.kernel_eval_count += K_test.size
        scores = self.svc.decision_function(K_test)
        self.inference_time_s = time.perf_counter() - t0
        return scores
```

## src/evaluation.py

```
"""Metrics for imbalanced clinical classification. Positive class = 1 (malignant)."""
import numpy as np
from sklearn.metrics import (roc_auc_score, average_precision_score, confusion_matrix,
                             precision_score, recall_score, f1_score, balanced_accuracy_score)


def scores_for(model, X):
    """Continuous scores for ranking metrics: predict_proba if available, else decision_function."""
    if hasattr(model, "predict_proba"):
        return model.predict_proba(X)[:, 1]
    return model.decision_function(X)


def compute_metrics(y_true, y_score, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    return {
        "auroc": roc_auc_score(y_true, y_score) if len(np.unique(y_true)) > 1 else np.nan,
        "auprc": average_precision_score(y_true, y_score),
        "sensitivity": recall_score(y_true, y_pred, zero_division=0),
        "specificity": tn / (tn + fp) if (tn + fp) else np.nan,
        "precision": precision_score(y_true, y_pred, zero_division=0),
```

```
        "f1": f1_score(y_true, y_pred, zero_division=0),
        "balanced_accuracy": balanced_accuracy_score(y_true, y_pred),
        "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp),
    }
```

## run_experiments.py

```
"""Runs the full comparison. Usage:  python run_experiments.py

For each of N_SEEDS random stratified 70/30 splits (same indices for every model):
  * Full-feature classical models (all 30 features): majority baseline, logistic
    regression, RBF SVM, random forest.
  * Matched 4-feature comparison (features chosen on TRAINING rows only):
    classical RBF SVM vs quantum kernel SVM (ZZFeatureMap, 4 qubits).
  * Repeated for training-set fractions 10%, 25%, 50%, 100% (stratified subsample).
All hyperparameters are chosen by CV on training rows. The test set is used only once.
"""
import time, os, json
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

import config as C
from src.data_loader import load_clinical_data
from src.preprocessing import select_top_k, quantum_angles
from src.classical_models import (majority_baseline, fit_logistic, fit_rbf_svm, fit_random_forest)
from src.quantum_model import FastQuantumKernelSVM, build_quantum_kernel, circuit_resources
from src.evaluation import compute_metrics, scores_for

OUT = os.path.join(os.path.dirname(__file__), "results")
os.makedirs(OUT, exist_ok=True)


def tune_quantum_C(K_train, y_train, seed):
    """Choose SVM C for the quantum kernel by stratified CV on the training kernel only."""
    best_C, best_score = None, -np.inf
    cv = StratifiedKFold(C.CV_FOLDS, shuffle=True, random_state=seed)
    for c in [0.1, 1.0, 10.0]:
        scores = []
        for tr, va in cv.split(K_train, y_train):
            svc = SVC(kernel="precomputed", C=c).fit(K_train[np.ix_(tr, tr)], y_train[tr])
            s = svc.decision_function(K_train[np.ix_(va, tr)])
            from sklearn.metrics import average_precision_score
            scores.append(average_precision_score(y_train[va], s))
        if np.mean(scores) > best_score:
            best_C, best_score = c, np.mean(scores)
    return best_C


def run_one(model_name, fit_fn, X_tr, y_tr, X_te, y_te):
    """Fit, time, evaluate one model. Returns a dict row."""
    t0 = time.perf_counter()
    model = fit_fn()
    fit_time = time.perf_counter() - t0
    t1 = time.perf_counter()
    scores = scores_for(model, X_te)
    y_pred = model.predict(X_te)
    infer_time = time.perf_counter() - t1
    row = compute_metrics(y_te, scores, y_pred)
    row.update({"model": model_name, "fit_time_s": fit_time, "inference_time_s": infer_time})
    return row, scores, y_pred


def main():
    X, y, feature_names = load_clinical_data()
    all_rows, log = [], []
    pred_store = {}
    for seed in range(C.N_SEEDS):
        X_tr_full, X_te, y_tr_full, y_te = train_test_split(
            X, y, test_size=C.TEST_SIZE, stratify=y, random_state=seed)
        for frac in C.TRAIN_FRACTIONS:
            if frac < 1.0:
                # stratified subsample of TRAINING rows only; test set unchanged
                X_tr, _, y_tr, _ = train_test_split(
                    X_tr_full, y_tr_full, train_size=frac, stratify=y_tr_full, random_state=seed)
            else:
                X_tr, y_tr = X_tr_full, y_tr_full
            n_tr = len(y_tr)

            # ---- full-feature classical models (scaler fitted on training rows) ----
            sc = StandardScaler().fit(X_tr)
            Xs_tr, Xs_te = sc.transform(X_tr), sc.transform(X_te)
            full_models = {
                "classical_majority_baseline": lambda: majority_baseline().fit(Xs_tr, y_tr),
                "classical_logistic_30feat": lambda: fit_logistic(Xs_tr, y_tr, seed, C.CV_FOLDS),
                "classical_rbf_svm_30feat": lambda: fit_rbf_svm(Xs_tr, y_tr, seed, C.CV_FOLDS),
                "classical_random_forest_30feat": lambda: fit_random_forest(X_tr, y_tr, seed, C.CV_FOLDS),
            }
            for name, fn in full_models.items():
                if name == "classical_random_forest_30feat":
                    row, sc_, yp = run_one(name, fn, X_tr, y_tr, X_te, y_te)
```

```
                else:
                    row, sc_, yp = run_one(name, fn, Xs_tr, y_tr, Xs_te, y_te)
                row.update({"seed": seed, "train_fraction": frac, "n_train": n_tr,
                            "n_features": 30, "n_test": len(y_te)})
                all_rows.append(row)
                if seed == 0 and frac == 1.0:
                    pred_store[name] = (y_te, sc_, yp)

            # ---- matched 4-feature comparison (selection on training rows only) ----
            sel = select_top_k(X_tr, y_tr, C.N_QUBITS_FEATURES)
            Xk_tr, Xk_te = sel.transform(X_tr), sel.transform(X_te)
            sc4 = StandardScaler().fit(Xk_tr)
            Xk_tr_s, Xk_te_s = sc4.transform(Xk_tr), sc4.transform(Xk_te)
            row, sc_, yp = run_one("classical_rbf_svm_4feat", lambda: fit_rbf_svm(Xk_tr_s, y_tr, seed, C.CV_FOLDS),
                                   Xk_tr_s, y_tr, Xk_te_s, y_te)
            row.update({"seed": seed, "train_fraction": frac, "n_train": n_tr,
                        "n_features": C.N_QUBITS_FEATURES, "n_test": len(y_te)})
            all_rows.append(row)

            # quantum: angles from TRAINING range only
            ang_tr, ang_te = quantum_angles(Xk_tr_s, Xk_te_s)
            fmap, _ = build_quantum_kernel(C.N_QUBITS_FEATURES)
            t_k0 = time.perf_counter()
            from src.quantum_model import exact_fidelity_kernel
            K_train = exact_fidelity_kernel(fmap, ang_tr)
            kernel_time = time.perf_counter() - t_k0
            q_C = tune_quantum_C(K_train, y_tr, seed)

            def fit_quantum():
                qm = FastQuantumKernelSVM(C.N_QUBITS_FEATURES, C=q_C, seed=seed)
                return qm.fit(ang_tr, y_tr)

            t0 = time.perf_counter()
            qmodel = fit_quantum()
            fit_time = time.perf_counter() - t0 + kernel_time * 0  # fit includes kernel build
            t1 = time.perf_counter()
            q_scores = qmodel.decision_function(ang_te)
            infer_time = time.perf_counter() - t1
            q_pred = (q_scores > 0).astype(int)
            row = compute_metrics(y_te, q_scores, q_pred)
            res = circuit_resources(fmap)
            row.update({"model": "quantum_kernel_svm_4qubit", "fit_time_s": fit_time,
                        "inference_time_s": infer_time, "seed": seed, "train_fraction": frac,
                        "n_train": n_tr, "n_features": C.N_QUBITS_FEATURES, "n_test": len(y_te),
                        "C_selected": q_C, "qubits": res["qubits"],
                        "circuit_depth": res["circuit_depth"],
                        "two_qubit_gates": res["two_qubit_gates"],
                        "kernel_entries_computed": qmodel.kernel_eval_count,
                        "train_kernel_shape": f"{qmodel.kernel_matrix_shape[0]}x{qmodel.kernel_matrix_shape[1]}",
                        "shots": "exact statevector (no shots)"})
            all_rows.append(row)
            if seed == 0 and frac == 1.0:
                pred_store["quantum_kernel_svm_4qubit"] = (y_te, q_scores, q_pred)
            print(f"seed={seed} frac={frac} n_train={n_tr} done")
    df = pd.DataFrame(all_rows)
    df.to_csv(os.path.join(OUT, "metrics.csv"), index=False)
    settings = {"random_seed_base": 0, "n_seeds": C.N_SEEDS, "test_size": C.TEST_SIZE,
                "train_fractions": C.TRAIN_FRACTIONS, "cv_folds": C.CV_FOLDS,
                "primary_metric": C.PRIMARY_METRIC, "n_qubits": C.N_QUBITS_FEATURES,
                "quantum_feature_map": "ZZFeatureMap reps=2 linear entanglement",
                "quantum_simulation": "exact noiseless statevector (no hardware, no shot noise)",
                "feature_selection": "top-k ANOVA F-score on training rows",
                "dataset": "sklearn breast_cancer (bundled), 569 rows, positive=malignant"}
    with open(os.path.join(OUT, "experiment_log.json"), "w") as f:
        json.dump(settings, f, indent=2)
    pd.DataFrame(list(settings.items()), columns=["setting", "value"]).astype(str).to_csv(
        os.path.join(OUT, "experiment_log.csv"), index=False)
    # predictions for the figure/confusion matrix (seed 0, full training set)
    import pickle
    with open(os.path.join(OUT, "predictions_seed0_full_train.pkl"), "wb") as f:
        pickle.dump(pred_store, f)
    print("saved", len(df), "rows")


if __name__ == "__main__":
    main()
```

## run_ablation.py

```
"""Ablation: quantum circuit configurations (100% training set, same 5 seeds/splits).
Configs: (qubits, reps). Feature selection and C-tuning follow the main experiment.
Writes results/ablation_quantum.csv
"""
import os, numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import config as C
from src.data_loader import load_clinical_data
from src.preprocessing import select_top_k, quantum_angles
from src.quantum_model import exact_fidelity_kernel, build_quantum_kernel, circuit_resources, FastQuantumKernelSVM
from src.evaluation import compute_metrics
from run_experiments import tune_quantum_C

OUT = os.path.join(os.path.dirname(__file__), "results")
```

```
CONFIGS = [(2, 1), (4, 1), (4, 2), (6, 2)]  # (qubits, reps)

X, y, _ = load_clinical_data()
rows = []
for seed in range(C.N_SEEDS):
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=C.TEST_SIZE, stratify=y, random_state=seed)
    for nq, reps in CONFIGS:
        sel = select_top_k(Xtr, ytr, nq)
        a, b = sel.transform(Xtr), sel.transform(Xte)
        s = StandardScaler().fit(a)
        ang_tr, ang_te = quantum_angles(s.transform(a), s.transform(b))
        fmap, _ = build_quantum_kernel(nq, reps=reps)
        K = exact_fidelity_kernel(fmap, ang_tr)
        off = K[~np.eye(len(K), dtype=bool)]
        qC = tune_quantum_C(K, ytr, seed)
        qm = FastQuantumKernelSVM(nq, C=qC, reps=reps, seed=seed).fit(ang_tr, ytr)
        sc = qm.decision_function(ang_te)
        m = compute_metrics(yte, sc, (sc > 0).astype(int))
        res = circuit_resources(fmap)
        rows.append({"seed": seed, "qubits": nq, "reps": reps, "C": qC,
                     "mean_offdiag_kernel": off.mean(), "std_offdiag_kernel": off.std(),
                     "circuit_depth": res["circuit_depth"], "two_qubit_gates": res["two_qubit_gates"],
                     **m})
    print("seed", seed, "done", flush=True)
df = pd.DataFrame(rows)
df.to_csv(os.path.join(OUT, "ablation_quantum.csv"), index=False)
print(df.groupby(["qubits", "reps"])[["auroc", "auprc", "sensitivity", "specificity", "mean_offdiag_kernel",
                                      "circuit_depth"]].agg(["mean", "std"]).round(3).to_string())
```

## make_figures.py

```
"""Creates the four required figures in results/figures/ from saved results."""
import os, pickle
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

R = os.path.join(os.path.dirname(__file__), "results")
F = os.path.join(R, "figures"); os.makedirs(F, exist_ok=True)
df = pd.read_csv(os.path.join(R, "metrics.csv"))
NAMES = {"classical_majority_baseline": "Majority baseline",
         "classical_logistic_30feat": "Logistic (30 feat.)",
         "classical_rbf_svm_30feat": "RBF SVM (30 feat.)",
         "classical_random_forest_30feat": "Random forest (30 feat.)",
         "classical_rbf_svm_4feat": "RBF SVM (4 feat.)",
         "quantum_kernel_svm_4qubit": "Quantum kernel SVM (4 qubits)"}
ORDER = list(NAMES)
COLORS = {"classical_majority_baseline": "#9e9e9e", "classical_logistic_30feat": "#1f77b4",
          "classical_rbf_svm_30feat": "#2ca02c", "classical_random_forest_30feat": "#8c564b",
          "classical_rbf_svm_4feat": "#17becf", "quantum_kernel_svm_4qubit": "#d62728"}

# Figure 1: model comparison at 100% training data (mean +/- sd over 5 splits)
full = df[df.train_fraction == 1.0].groupby("model")
stats = full[["auprc", "auroc"]].agg(["mean", "std"])
fig, ax = plt.subplots(figsize=(9, 4.8))
x = np.arange(len(ORDER))
ax.bar(x - 0.2, [stats.loc[m, ("auprc", "mean")] for m in ORDER], 0.4,
       yerr=[stats.loc[m, ("auprc", "std")] for m in ORDER], capsize=3, label="AUPRC (primary)",
       color=[COLORS[m] for m in ORDER])
ax.bar(x + 0.2, [stats.loc[m, ("auroc", "mean")] for m in ORDER], 0.4,
       yerr=[stats.loc[m, ("auroc", "std")] for m in ORDER], capsize=3, label="AUROC", alpha=0.45,
       color=[COLORS[m] for m in ORDER])
ax.set_xticks(x); ax.set_xticklabels([NAMES[m] for m in ORDER], rotation=25, ha="right")
ax.set_ylim(0.4, 1.02); ax.set_ylabel("Score (higher is better)")
ax.set_title("Model comparison, 100% of training data (mean ± SD, 5 random splits)")
ax.legend(loc="lower right"); fig.tight_layout()
fig.savefig(os.path.join(F, "fig1_model_comparison.png"), dpi=150); plt.close(fig)

# Figure 2: training-set size vs AUPRC
fig, ax = plt.subplots(figsize=(8, 4.8))
for m in ORDER:
    if m == "classical_majority_baseline":
        continue
    g = df[df.model == m].groupby("train_fraction")["auprc"].agg(["mean", "std"])
    ax.errorbar(g.index * 100, g["mean"], yerr=g["std"], marker="o", capsize=3,
                label=NAMES[m], color=COLORS[m], lw=2 if "quantum" in m else 1.3)
ax.set_xlabel("Training data used (% of 398 training rows)"); ax.set_ylabel("AUPRC (test set)")
ax.set_xticks([10, 25, 50, 100]); ax.set_ylim(0.6, 1.0)
ax.set_title("Training-set size vs performance (mean ± SD, 5 splits)")
ax.legend(fontsize=8, loc="lower right"); ax.grid(alpha=0.3); fig.tight_layout()
fig.savefig(os.path.join(F, "fig2_training_size.png"), dpi=150); plt.close(fig)

# Figure 3: resource / execution cost at 100% (fit and inference time, log scale)
fig, ax = plt.subplots(figsize=(9, 4.8))
rs = full[["fit_time_s", "inference_time_s"]].mean()
y = np.arange(len(ORDER))
ax.barh(y + 0.2, [rs.loc[m, "fit_time_s"] for m in ORDER], 0.4, label="Fit time (s, incl. CV tuning for classical)")
ax.barh(y - 0.2, [rs.loc[m, "inference_time_s"] for m in ORDER], 0.4, label="Inference time (s, test set)")
ax.set_xscale("log"); ax.set_yticks(y); ax.set_yticklabels([NAMES[m] for m in ORDER])
ax.set_xlabel("Seconds (log scale), mean over 5 splits"); ax.legend(fontsize=8, loc="lower right")
ax.set_title("Execution cost at 100% training data")
```

```
fig.tight_layout(); fig.savefig(os.path.join(F, "fig3_resource_cost.png"), dpi=150); plt.close(fig)

# Figure 4: confusion matrices (seed 0, full training set), final selected model + quantum
with open(os.path.join(R, "predictions_seed0_full_train.pkl"), "rb") as f:
    preds = pickle.load(f)
final = "classical_logistic_30feat"  # best classical model by mean AUPRC at 100% (pre-declared rule)
fig, axes = plt.subplots(1, 2, figsize=(9, 4))
for ax, m in zip(axes, [final, "quantum_kernel_svm_4qubit"]):
    yt, _, yp = preds[m]
    cm = confusion_matrix(yt, yp, labels=[0, 1])
    ax.imshow(cm, cmap="Blues")
    for i in range(2):
        for j in range(2):
            ax.text(j, i, cm[i, j], ha="center", va="center", fontsize=14,
                    color="white" if cm[i, j] > cm.max() / 2 else "black")
    ax.set_xticks([0, 1]); ax.set_yticks([0, 1])
    ax.set_xticklabels(["Benign", "Malignant"]); ax.set_yticklabels(["Benign", "Malignant"])
    ax.set_xlabel("Predicted"); ax.set_ylabel("True"); ax.set_title(NAMES[m], fontsize=10)
fig.suptitle("Confusion matrices, test set, seed 0, 100% training data", fontsize=11)
fig.tight_layout(); fig.savefig(os.path.join(F, "fig4_confusion_matrix.png"), dpi=150); plt.close(fig)
print("figures written:", sorted(os.listdir(F)))
```

## build_pdf.py

```
"""Builds the final submission PDF from results files. Usage: python build_pdf.py"""
import os
import pandas as pd
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib import colors
from reportlab.lib.units import cm
from reportlab.platypus import (SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle,
                                Image, PageBreak)

BASE = os.path.dirname(os.path.abspath(__file__))
R = os.path.join(BASE, "results"); FIG = os.path.join(R, "figures")
OUT = os.path.join(BASE, "report", "submission.pdf")
df = pd.read_csv(os.path.join(R, "metrics.csv"))
abl = pd.read_csv(os.path.join(R, "ablation_quantum.csv"))
NAMES = {"classical_majority_baseline": "Majority baseline",
         "classical_logistic_30feat": "Logistic regression (30 features)",
         "classical_rbf_svm_30feat": "RBF SVM (30 features)",
         "classical_random_forest_30feat": "Random forest (30 features)",
         "classical_rbf_svm_4feat": "RBF SVM (same 4 features)",
         "quantum_kernel_svm_4qubit": "Quantum kernel SVM (4 qubits)"}

ss = getSampleStyleSheet()
NAVY = colors.HexColor("#1B3A5C"); TEAL = colors.HexColor("#2A9D8F"); CORAL = colors.HexColor("#D1495B")
PALE = colors.HexColor("#EEF4FA"); AMBER_BG = colors.HexColor("#FFF6E5"); AMBER = colors.HexColor("#E09F3E")
GREY_TXT = colors.HexColor("#5A6470"); CODE_BG = colors.HexColor("#F4F6F8")
H1 = ParagraphStyle("H1", parent=ss["Heading1"], fontName="Helvetica-Bold", fontSize=15, leading=19,
                    textColor=NAVY, spaceBefore=14, spaceAfter=7)
H2 = ParagraphStyle("H2", parent=ss["Heading2"], fontName="Helvetica-Bold", fontSize=11.5, leading=15,
                    textColor=TEAL, spaceBefore=9, spaceAfter=4)
B = ParagraphStyle("B", parent=ss["BodyText"], fontName="Helvetica", fontSize=9.5, leading=14, spaceAfter=6,
                   textColor=colors.HexColor("#22272E"))
SM = ParagraphStyle("SM", parent=B, fontSize=8, leading=11, textColor=GREY_TXT, spaceAfter=4)
T = ParagraphStyle("T", parent=ss["Title"], fontName="Helvetica-Bold", fontSize=22, leading=27,
                   textColor=NAVY, alignment=0, spaceAfter=8)

def P(t, s=B): return Paragraph(t, s)

def ms(model, frac, col):
    g = df[(df.model == model) & (df.train_fraction == frac)][col]
    return f"{g.mean():.3f} ± {g.std():.3f}"

def table(rows, widths, header=True):
    t = Table(rows, colWidths=widths, repeatRows=1 if header else 0)
    style = [("FONT", (0, 0), (-1, -1), "Helvetica", 7.8),
             ("TEXTCOLOR", (0, 0), (-1, -1), colors.HexColor("#22272E")),
             ("GRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#C9D3DF")),
             ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
             ("TOPPADDING", (0, 0), (-1, -1), 4), ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
             ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, PALE])]
    if header:
        style += [("BACKGROUND", (0, 0), (-1, 0), NAVY),
                  ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
                  ("FONT", (0, 0), (-1, 0), "Helvetica-Bold", 7.8)]
    t.setStyle(TableStyle(style)); return t

story = []
# ---------------- Title / abstract ----------------
story += [P("Quantum Kernel versus Classical Classifiers for Clinical Malignancy Prediction", T),
          P("Educational research prototype. Not a medical device. Not for clinical use.", SM), Spacer(1, 8),
          P("<b>Abstract.</b> We compared a 4-qubit quantum kernel support vector machine (SVM), simulated exactly "
            "without hardware noise, with classical classifiers on the Breast Cancer Wisconsin (Diagnostic) dataset "
            "(569 patients, 30 numeric features, 212 malignant cases). We used five repeated stratified 70/30 splits, "
            "training-only preprocessing and feature selection, cross-validated hyperparameters, and four training-set "
            "sizes (10, 25, 50 and 100 per cent). The best classical model (logistic regression on all 30 features) "
            "reached AUPRC 0.991 ± 0.005 on the held-out test sets. The quantum model reached 0.932 ± 0.013 at full "
            "training size. It was also below a classical RBF SVM given the same four features (0.978 ± 0.008). "
            "The quantum model did not outperform classical methods in any condition tested. Because this is a "
            "simulation on a small, well-studied dataset, the results do not establish quantum advantage."),
```

```
          P("<b>Important change from the original plan.</b> The original target was diabetic retinopathy using the "
            "Dryad dataset doi:10.5061/dryad.jwstqjqrm. We could not download that dataset: the sandbox proxy refused "
            "the connection to datadryad.org (HTTP 403, repeated). We therefore switched to a clinical dataset that "
            "ships inside the scikit-learn package, which works offline. The diabetic retinopathy experiment has "
            "<b>not</b> been run and no diabetic retinopathy results are reported here."),
          PageBreak()]

# ---------------- 1 Problem ----------------
story += [P("1. Clinical problem and why it matters", H1),
          P("Clinicians often use numerical measurements from a biopsy or scan to judge whether a tumour is malignant "
            "(cancerous). Missing a malignant case (a false negative) can delay treatment, so sensitivity matters. "
            "A model that simply predicts the common class is useless, which is why we report precision-recall "
            "measures and not accuracy alone. This project is a methods study. Its output is a classifier score, and "
            "it does not diagnose anyone or replace a clinician."),
          P("<b>Dataset.</b> Breast Cancer Wisconsin (Diagnostic), bundled with scikit-learn. It is derived from "
            "digitised images of fine-needle aspirate cell nuclei. Thirty features describe the mean, standard error "
            "and worst value of radius, texture, perimeter, area, smoothness and related shape measures. "
            "Labels: malignant (positive class, 212 cases) and benign (357 cases). This is a well-known public "
            "benchmark, first described by Street, Wolberg and Mangasarian (1993). Its licence should be confirmed "
            "from the original UCI page before public release. This report does not assert the licence terms."),
          P("<b>Limitations of the dataset.</b> It is small (569 rows), comes from a single source, and has no "
            "demographic data, so fairness across groups cannot be assessed. It is not representative of routine "
            "screening populations, and it contains already-curated image-derived features rather than raw records.")]

# ---------------- 2 Research question ----------------
story += [P("2. Research question, hypotheses and outcomes", H1),
          P("<b>Primary question.</b> With controlled training-data sizes, can a 4-qubit quantum kernel SVM match or "
            "outperform strong classical classifiers in classifying malignant cases on held-out data?"),
          P("<b>H1.</b> The quantum kernel SVM matches or exceeds a classical RBF SVM given the same four features "
            "(primary outcome: AUPRC on the test set, 100% training data). <b>Result: not supported.</b> "
            "0.932 ± 0.013 versus 0.978 ± 0.008."),
          P("<b>H2.</b> The quantum model's advantage is largest when training data are scarce. <b>Result: not "
            "supported.</b> At 10% training data it reached 0.737 ± 0.118, against 0.946 ± 0.047 for the matched "
            "classical SVM."),
          P("<b>H3.</b> The quantum model stays competitive on ranking metrics (AUROC) while its operating point is "
            "less stable. <b>Result: partly supported.</b> AUROC was 0.953 ± 0.013 at 100% data, close to the "
            "matched classical SVM at 0.982 ± 0.007, but sensitivity was unstable (0.731 ± 0.227).")]

# ---------------- 3 Methods ----------------
story += [P("3. Methods", H1),
          P("<b>Splits.</b> For each of five seeds (0 to 4), a stratified 70/30 split produced 398 training and 171 "
            "test rows. Every model used the same indices. Training subsets at 10, 25 and 50 per cent were drawn by "
            "stratified sampling from the training rows only. The test set never changed."),
          P("<b>Leakage control.</b> The standard scaler, feature selector (ANOVA F-test, top-k) and quantum angle "
            "range were all fitted on training rows only. Test values outside the training range were clipped to "
            "[0, π]. Unit tests in tests/test_core.py check this behaviour."),
          P("<b>Classical models.</b> A majority-class baseline; logistic regression (C ∈ {0.01, 0.1, 1, 10}); an RBF "
            "SVM (C ∈ {0.1, 1, 10}, gamma ∈ {scale, 0.01, 0.1}); a random forest (300 trees, max depth and minimum "
            "leaf size tuned). Hyperparameters were chosen by 5-fold stratified cross-validation on the training rows, "
            "scored by average precision."),
          P("<b>Quantum model.</b> The four features with the highest training-set F-score were angle-encoded into "
            "four qubits using a ZZFeatureMap (2 repetitions, linear entanglement). The fidelity kernel "
            "K(x, x′) = |■ψ(x)|ψ(x′)■|² was computed as the exact statevector overlap. "
            "Qiskit's own kernel was checked "
            "against this at high shot counts (tests/test_core.py). The SVM regularisation constant C was chosen by "
            "cross-validation on the training kernel. The circuit configuration (4 qubits, 2 repetitions) was fixed "
            "before the main experiment was run."),
          P("<b>Metrics.</b> AUROC, AUPRC (primary), sensitivity, specificity, precision, F1, balanced accuracy and "
            "confusion-matrix counts. Mean ± standard deviation over five splits. The decision threshold is zero for "
            "the SVMs and 0.5 probability for logistic regression. Thresholds were not tuned on test data.")]

# ---------------- 4 Results ----------------
story += [P("4. Results", H1), P("<b>Table 1. Full training data (100%), test set, mean ± SD over 5 splits.</b>", SM)]
rows = [["Model", "AUROC", "AUPRC", "Sensitivity", "Specificity", "F1", "Fit time (s)", "Inference (s)"]]
for m in NAMES:
    g = df[(df.model == m) & (df.train_fraction == 1.0)]
    rows.append([NAMES[m], ms(m, 1.0, "auroc"), ms(m, 1.0, "auprc"), ms(m, 1.0, "sensitivity"),
                 ms(m, 1.0, "specificity"), ms(m, 1.0, "f1"),
                 f"{g.fit_time_s.mean():.3f}", f"{g.inference_time_s.mean():.4f}"])
story += [table(rows, [4.6*cm, 2*cm, 2*cm, 2*cm, 2*cm, 1.7*cm, 1.9*cm, 1.9*cm]), Spacer(1, 8)]

story += [P("<b>Table 2. AUPRC by training-set size (test set, mean ± SD).</b>", SM)]
rows = [["Model", "10%", "25%", "50%", "100%"]]
for m in NAMES:
    if m == "classical_majority_baseline":
        continue
    rows.append([NAMES[m]] + [ms(m, f, "auprc") for f in [0.10, 0.25, 0.50, 1.00]])
story += [table(rows, [5.6*cm, 2.8*cm, 2.8*cm, 2.8*cm, 2.8*cm]), Spacer(1, 8)]

story += [P("<b>Table 3. Confusion matrix counts, summed over five splits, 100% training data.</b>", SM)]
rows = [["Model", "True negatives", "False positives", "False negatives", "True positives"]]
for m in NAMES:
    g = df[(df.model == m) & (df.train_fraction == 1.0)]
    rows.append([NAMES[m]] + [str(int(g[c].sum())) for c in ["tn", "fp", "fn", "tp"]])
story += [table(rows, [5.6*cm, 2.8*cm, 2.8*cm, 2.8*cm, 2.8*cm]), Spacer(1, 6),
          P("Test sets contained 107 benign and 64 malignant cases per split on average (171 in total). "
            "The majority baseline predicts every case as benign, so it misses all malignant cases.", SM)]

story += [PageBreak(), P("<b>Figures.</b>", H2),
          Image(os.path.join(FIG, "fig1_model_comparison.png"), width=16*cm, height=8.5*cm),
          Image(os.path.join(FIG, "fig2_training_size.png"), width=14*cm, height=8.4*cm),
          PageBreak(),
          Image(os.path.join(FIG, "fig3_resource_cost.png"), width=16*cm, height=8.5*cm),
          Image(os.path.join(FIG, "fig4_confusion_matrix.png"), width=16*cm, height=7.1*cm)]
```

```
# ---------------- 5 Quantum resources ----------------
q = df[df.model == "quantum_kernel_svm_4qubit"]
story += [PageBreak(), P("5. Quantum resource and efficiency analysis", H1),
          P("<b>Measured (from the circuit object, not estimated):</b> 4 qubits; circuit depth 19 after decomposition; "
            "12 two-qubit gates per feature-map circuit; 34 total gates. Kernel entries computed per run: 8,190 (10%), "
            "26,730 (25%), 73,630 (50%) and 226,462 (100%, a 398×398 training kernel plus a 171×398 test kernel). "
            "Each entry is one exact overlap calculation. The simulator used exact statevectors, so no shots were "
            "taken and there was no hardware noise. Hardware runs would need shot counts and error mitigation that "
            "this project did not include."),
          P("<b>Timing:</b> the quantum fit time (0.254 ± 0.002 s at 100%) includes kernel construction and SVM fitting, "
            "but <b>excludes</b> the cross-validation used to choose C. Classical fit times include their "
            "cross-validation. The two fit times are therefore not directly comparable. Timings are CPU wall-clock times "
            "on the sandbox machine, with no hardware."),
          P("<b>Scaling (theoretical, not measured):</b> a statevector on n qubits has 2^n amplitudes, so exact "
            "simulation memory grows exponentially with qubit count. Our 6-qubit ablation kept the kernel "
            "computation tractable, but larger feature sets would need approximate or hardware methods. The number of "
            "kernel evaluations grows with the square of training size, which is why the quantum model was limited to "
            "a few hundred training rows here.")]

# ---------------- 6 Ablation ----------------
ab = abl.groupby(["qubits", "reps"]).agg(
    auprc_m=("auprc", "mean"), auprc_s=("auprc", "std"), auroc_m=("auroc", "mean"),
    sens_m=("sensitivity", "mean"), sens_s=("sensitivity", "std"),
    off=("mean_offdiag_kernel", "mean"), depth=("circuit_depth", "mean"), gates=("two_qubit_gates", "mean")).reset_index()
rows = [["Qubits", "Repetitions", "AUPRC", "AUROC", "Sensitivity", "Mean off-diag. kernel", "Circuit depth",
         "2-qubit gates"]]
for _, r in ab.iterrows():
    rows.append([int(r.qubits), int(r.reps), f"{r.auprc_m:.3f} ± {r.auprc_s:.3f}", f"{r.auroc_m:.3f}",
                 f"{r.sens_m:.3f} ± {r.sens_s:.3f}", f"{r.off:.3f}", f"{r.depth:.0f}", f"{r.gates:.0f}"])
story += [P("6. Ablation of quantum circuit configurations (post hoc)", H1),
          P("These four configurations were run after the main experiment, at 100% training data over the same five "
            "seeds. The main results above keep the configuration fixed in advance (4 qubits, 2 repetitions). "
            "The ablation is <b>exploratory</b>: picking the best row here would be choosing on the test set. Its purpose "
            "is to show how the kernel behaves, not to replace the main result."),
          P("<b>Table 4.</b>", SM), table(rows, [1.6*cm, 2.2*cm, 2.8*cm, 1.9*cm, 2.8*cm, 3.2*cm, 2.1*cm, 2.1*cm]),
          Spacer(1, 6),
          P("<b>Interpretation.</b> With 2 qubits, the kernel values are large and similar (mean off-diagonal ≈ 0.35), "
            "and the model is weakest. With 6 qubits the average kernel value falls to ≈ 0.05. This is a known "
            "pattern called kernel concentration: as the encoding becomes more complex, most pairs of patients look "
            "almost equally dissimilar, so the SVM sees less structure. The two most accurate configurations were "
            "4 qubits with 1 repetition and 6 qubits with 2 repetitions, which were similar on AUPRC (about 0.95) and "
            "AUROC (about 0.96). The 4-qubit, 1-repetition configuration had the highest and most stable sensitivity. "
            "These configurations were compared after the test results were visible, so they are not validated model "
            "choices, and the main result keeps the pre-specified configuration.")]

# ---------------- 7 Discussion ----------------
story += [P("7. Discussion: what the results do and do not establish", H1),
          P("<b>Established by this study:</b> on this dataset, with exact simulation, a 4-qubit fidelity kernel SVM "
            "was less accurate than classical SVMs and logistic regression given equal or comparable inputs. Its errors "
            "were concentrated in false negatives (17 at 100% data, against 6 for logistic regression), which is the "
            "clinically worse direction."),
          P("<b>Not established:</b> that quantum methods are generally inferior, that quantum advantage is impossible, "
            "or anything about hardware performance. The experiment used a small, well-studied dataset in which "
            "classical models already reach AUPRC above 0.97. There is little room for a quantum method to help, and "
            "small datasets cannot show general advantage. Quantum speedup, quantum advantage and clinical readiness "
            "are not claimed."),
          P("<b>Why the quantum model may underperform.</b> (a) Only four features were used, so the circuit compresses "
            "the classical information. (b) The ZZ feature map is one of many possible designs, and the ablation shows "
            "that its kernel values concentrate as qubits increase. (c) Noiseless simulation removes hardware noise, "
            "which means the result is the best case for the quantum circuit, not a realistic one. (d) The classical "
            "SVM and logistic regression capture most of the signal in the full feature set.")]

# ---------------- 8 Limitations ----------------
story += [P("8. Limitations", H1),
          P("• <b>Not the planned dataset.</b> No diabetic retinopathy results. The clinical question is different from "
            "the one originally proposed."),
          P("• <b>Single small dataset, no external validation</b>. Temporal or external validation was not possible "
            "for this dataset."),
          P("• <b>Threshold-dependent sensitivity.</b> SVM decision scores are thresholded at zero, which is not tuned. "
            "The quantum kernel's sensitivity varies widely between splits (SD 0.227 at 100%; 0.421 at 25%). "
            "This should be fixed with a validation-based threshold in a future study."),
          P("• <b>Exploratory ablation.</b> The configuration comparison was run after seeing test results."),
          P("• <b>Simulation only.</b> No IBM Quantum hardware, no noise models and no shot-based estimation."),
          P("• <b>Unequal timing methodology.</b> As noted in Section 5, fit times are not directly comparable."),
          P("• <b>Sample size.</b> Five splits give limited evidence about variance. Confidence intervals are not "
            "reported beyond SD."),
          P("• <b>Competition rules were not verified.</b> The web search for the Khalifa University and IBM challenge "
            "returned unrelated results, so word limits, formats and judging criteria are unknown.")]

# ---------------- 9 Ethics ----------------
story += [P("9. Ethical considerations", H1),
          P("The dataset is de-identified and public, but whether its original collection met current consent standards "
            "is not documented here. Models built on a small, single-source dataset can be biased against groups "
            "absent from it. A false negative means a missed cancer, so any future clinical version would need "
            "clinical trials, subgroup analyses and regulatory review. This project states none of these as complete. "
            "It is a classroom-scale educational prototype.")]

# ---------------- 10 Reproducibility ----------------
story += [P("10. Reproducibility", H1),
          P("<b>Environment:</b> Python 3.13, scikit-learn 1.9.1, pandas 3.0.5, qiskit 2.6.1, qiskit-machine-learning "
            "0.9.1, reportlab 5.0.1, matplotlib 3.11.2, pytest. All random seeds are fixed in config.py "
            "(five splits, seeds 0–4)."),
          P("<b>Commands:</b> <font face='Courier'>pip install scikit-learn pandas qiskit qiskit-machine-learning "
            "reportlab matplotlib pytest</font> then <font face='Courier'>python -m pytest tests</font>, "
            "<font face='Courier'>python run_experiments.py</font>, <font face='Courier'>python run_ablation.py</font>, "
```

```
            "<font face='Courier'>python make_figures.py</font> and <font face='Courier'>python build_pdf.py</font>."),
          P("<b>Tests:</b> five tests (labels and shape, training-only angle range, metric values, quantum kernel "
            "symmetry and bounds, and agreement with Qiskit's kernel at high shot count). All five passed in the "
            "sandbox."),
          P("<b>Source inventory:</b> config.py; src/data_loader.py, src/preprocessing.py, src/classical_models.py, "
            "src/quantum_model.py, src/evaluation.py; run_experiments.py; run_ablation.py; make_figures.py; "
            "build_pdf.py; tests/test_core.py; results/metrics.csv, results/ablation_quantum.csv, "
            "results/experiment_log.json and results/figures/*.png.")]

# ---------------- 11 Judge questions ----------------
story += [P("11. Anticipated judge questions", H1),
          P("<b>Why not diabetic retinopathy?</b> The planned dataset could not be downloaded from the sandbox. I used a "
            "public clinical dataset I could access, and I say so openly."),
          P("<b>Did the quantum model win?</b> No. It was below the matched classical SVM in every condition tested."),
          P("<b>What is a quantum kernel?</b> A similarity score between two patients, computed as the overlap of the "
            "quantum states their features produce. The SVM uses those similarities to draw a boundary."),
          P("<b>Why is it simulated?</b> Simulation gives exact results and avoids hardware noise. Real hardware would "
            "add noise and needs shot-based estimates, and this study did not test it."),
          P("<b>Could it help clinically?</b> Not on this evidence. Clinical use would require much larger data, "
            "external validation, subgroup analysis and regulatory approval."),
          P("<b>Why is sensitivity so unstable?</b> Thresholds were fixed at zero, not tuned. The ablation shows that "
            "kernel concentration makes the SVM's score distribution narrow, so small shifts flip many predictions.")]


# ---------------- Appendix: full source code ----------------
from reportlab.platypus import Preformatted
CODE_FILES = ["config.py", "src/data_loader.py", "src/preprocessing.py", "src/classical_models.py",
              "src/quantum_model.py", "src/evaluation.py", "run_experiments.py", "run_ablation.py",
              "make_figures.py", "build_pdf.py", "tests/test_core.py"]
CODE = ParagraphStyle("CODE", fontName="Courier", fontSize=6.3, leading=7.6, backColor=CODE_BG,
                      borderPadding=4, spaceBefore=2, spaceAfter=2)
story += [PageBreak(), P("Appendix A. Complete source code", H1),
          P("Every file needed to reproduce the results. Run from the project folder.", B)]
for fname in CODE_FILES:
    with open(os.path.join(BASE, fname)) as fh:
        text = fh.read()
    story += [P(f"<b>{fname}</b>", H2), Preformatted(text, CODE), Spacer(1, 6)]

def footer(canvas, doc):
    canvas.saveState(); canvas.setFont("Helvetica", 7.5)
    canvas.drawString(2*cm, 1.2*cm, "Educational research prototype. Not a medical device.")
    canvas.drawRightString(A4[0]-2*cm, 1.2*cm, f"Page {doc.page}")
    canvas.restoreState()

def boxed(flow, bg, edge):
    """Wrap a paragraph in a shaded box with a coloured left edge (styling only)."""
    t = Table([[flow]], colWidths=[17*cm])
    t.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, -1), bg),
                           ("LINEBEFORE", (0, 0), (0, -1), 3.2, edge),
                           ("LEFTPADDING", (0, 0), (-1, -1), 10), ("RIGHTPADDING", (0, 0), (-1, -1), 10),
                           ("TOPPADDING", (0, 0), (-1, -1), 8), ("BOTTOMPADDING", (0, 0), (-1, -1), 8)]))
    return t

for i, el in enumerate(story):
    if isinstance(el, Paragraph):
        txt = el.getPlainText()
        if txt.startswith("Abstract."):
            story[i] = boxed(el, PALE, TEAL)
        elif txt.startswith("Important change"):
            story[i] = boxed(el, AMBER_BG, AMBER)

# 5. title band on page 1, running header and footer on all pages
def decorate(canvas, doc):
    w, h = A4
    canvas.saveState()
    if doc.page == 1:
        canvas.setFillColor(NAVY); canvas.rect(0, h - 1.1*cm, w, 1.1*cm, stroke=0, fill=1)
        canvas.setFillColor(TEAL); canvas.rect(0, h - 1.22*cm, w, 0.12*cm, stroke=0, fill=1)
    else:
        canvas.setFillColor(NAVY); canvas.setFont("Helvetica-Bold", 7.5)
        canvas.drawString(2*cm, h - 1.2*cm, "Quantum vs classical clinical classifiers")
        canvas.setStrokeColor(colors.HexColor("#C9D3DF")); canvas.setLineWidth(0.5)
        canvas.line(2*cm, h - 1.45*cm, w - 2*cm, h - 1.45*cm)
    canvas.setStrokeColor(colors.HexColor("#C9D3DF")); canvas.setLineWidth(0.5)
    canvas.line(2*cm, 1.55*cm, w - 2*cm, 1.55*cm)
    canvas.setFillColor(GREY_TXT); canvas.setFont("Helvetica", 7.5)
    canvas.drawString(2*cm, 1.1*cm, "Educational research prototype. Not a medical device.")
    canvas.drawRightString(w - 2*cm, 1.1*cm, f"Page {doc.page}")
    canvas.restoreState()

os.makedirs(os.path.dirname(OUT), exist_ok=True)
SimpleDocTemplate(OUT, pagesize=A4, leftMargin=2*cm, rightMargin=2*cm, topMargin=1.8*cm,
                  bottomMargin=2*cm, title="Quantum vs classical clinical classifiers").build(
    story, onFirstPage=decorate, onLaterPages=decorate)
print("wrote", OUT)
```

## tests/test_core.py

```
"""Basic tests: data loading, leakage-safe preprocessing, metrics, quantum kernel shape."""
import sys, os
import numpy as np
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from src.data_loader import load_clinical_data
```

```
from src.preprocessing import quantum_angles
from src.evaluation import compute_metrics
from src.quantum_model import build_quantum_kernel


def test_labels_binary_and_shape():
    X, y, names = load_clinical_data()
    assert X.shape == (569, 30) and len(names) == 30
    assert set(np.unique(y)) == {0, 1}
    assert y.sum() == 212  # malignant count in the bundled dataset (positive class)


def test_angles_use_training_range_only():
    X_tr = np.array([[0.0], [10.0]])
    X_te = np.array([[-5.0], [20.0]])   # outside training range
    tr, te = quantum_angles(X_tr, X_te)
    assert tr.min() == 0 and np.isclose(tr.max(), np.pi)
    assert te.min() == 0 and np.isclose(te.max(), np.pi)  # clipped


def test_metrics_perfect_and_chance():
    y = np.array([0, 0, 1, 1])
    m = compute_metrics(y, np.array([0.1, 0.2, 0.8, 0.9]), np.array([0, 0, 1, 1]))
    assert m["auroc"] == 1.0 and m["sensitivity"] == 1.0 and m["specificity"] == 1.0
    assert m["tp"] == 2 and m["tn"] == 2


def test_quantum_kernel_is_symmetric_and_bounded():
    fmap, kernel = build_quantum_kernel(n_qubits=2, reps=1)
    X = np.random.default_rng(0).uniform(0, np.pi, size=(5, 2))
    K = kernel.evaluate(X)
    assert K.shape == (5, 5)
    assert np.allclose(K, K.T, atol=1e-8)
    assert (K >= -1e-9).all() and (K <= 1 + 1e-9).all()
    assert np.allclose(np.diag(K), 1.0, atol=1e-6)


def test_fast_kernel_matches_qiskit_kernel():
    """The exact statevector kernel must agree with qiskit-machine-learning's kernel."""
    from src.quantum_model import exact_fidelity_kernel
    from qiskit.primitives import StatevectorSampler
    from qiskit_machine_learning.state_fidelities import ComputeUncompute
    from qiskit_machine_learning.kernels import FidelityQuantumKernel
    from src.quantum_model import zz_feature_map
    fmap = zz_feature_map(3, reps=2, entanglement="linear")
    # Qiskit's kernel is SAMPLED (shot noise ~1/sqrt(shots)); use many shots so it
    # converges to the exact value we compare against.
    sampler = StatevectorSampler(default_shots=1_000_000)
    kernel = FidelityQuantumKernel(feature_map=fmap, fidelity=ComputeUncompute(sampler=sampler))
    X = np.random.default_rng(1).uniform(0, np.pi, size=(4, 3))
    assert np.allclose(kernel.evaluate(X), exact_fidelity_kernel(fmap, X), atol=5e-3)
```